CLINIC FRONT-DESK ASSISTANT

Detailed backend architecture | runtime and database connections

TARGET DESIGN

3 November 2026

Read this as the intended end-to-end wiring, not a claim that every API/workflow is already connected.

Patient chat client
Existing client; HTTP/JSON

HTTP

BACKEND APPLICATION | one Python / Uvicorn process

FastAPI application
Uvicorn entry point: app.main:app

API routers
Validate HTTP input/output

Application services
Coordinate business behavior

invoke graph

staff decision
save / load state

LangGraph workflow
Compiled graph runs in this backend process
Stable conversation ID maps to thread_id

resume

Staff approval handling
Record decision; resume same thread_id

approval

Emergency guard
Evaluated before ordinary routing

no emergency

Intent router
Only reached when not emergency

yes

appointments

clinic question

approval required

Safe emergency response
Immediate instructions; flag staff
Bypasses booking and RAG

Appointment flow
Search / booking behavior

Clinic information
Approved evidence only

Approval pause
Same-day / cancellation

application SQL

Repositories
Parameterized SQL; transaction boundaries
doctors / slots / bookings / approvals

psycopg connection
Application connection from
DATABASE_URL

LangGraph PostgresSaver
Separate checkpoint connection
Uses package migrations/setup

Staff client
Reviews pending approvals

Solid: request / data flow

Dashed: checkpoint, resume, or initialization

Amber: staff approval path

FastAPI returns the workflow result to the calling client.

RAG reads approved files; it does not query PostgreSQL.

No diagnosis or treatment recommendation is generated.

CONNECTED RESOURCES

Clinic-approved documents
Markdown/text files on disk; not patient records

LLM provider (real mode)
External HTTP request; timeout required

POSTGRESQL SERVICE
One database server; separate app and checkpoint paths

Application tables
doctors | appointment_slots
conversations | bookings | approvals
Constraints enforce reservation safety

checkpoint writes

LangGraph checkpoint tables
checkpoints | writes | blobs | migrations
Owned and initialized by PostgresSaver
Not duplicated in application schema

One-off database initializer
schema.sql + PostgresSaver.setup()

Architecture source: project docs/architecture.md. Diagram describes target wiring; see page 2 for implementation status and connection details.

HOW THE CONNECTIONS WORK

ARCHITECTURE NOTES

Request lifecycle, persistence boundaries, and what is implemented today

1 | Request and workflow lifecycle

The existing chat client sends HTTP/JSON to FastAPI. A route validates the request and calls an application service; FastAPI routes do not issue SQL directly.

The service invokes the compiled LangGraph workflow in the same Python/Uvicorn process. The graph checks for emergencies before ordinary intent routing.

Non-emergency requests branch to appointment handling, clinic-information retrieval, or an approval pause. FastAPI serializes the workflow result back to the client.

2 | Two PostgreSQL connection paths

Application data: services use repositories, which execute parameterized psycopg SQL using DATABASE_URL. The tables are doctors, appointment_slots, conversations, bookings, and approvals.

Workflow state: LangGraph's PostgresSaver uses its own PostgreSQL connection to the same database server. It stores checkpoints under the stable conversation thread_id.

The checkpoint schema is managed by the LangGraph package's setup/migrations. Do not recreate its internal checkpoint tables in schema.sql or access them through application repositories.

3 | Booking and staff approval safety

Booking creation must be atomic. PostgreSQL constraints protect against overlapping slots and competing active bookings, including concurrent requests; application code maps expected constraint failures to safe domain errors.

Same-day bookings and cancellations pause for staff approval. The approval record and booking state are updated transactionally, then the workflow resumes using the same thread_id.

A durable checkpoint allows the pending workflow to be restored after a backend restart. The staff decision must resume the matching conversation, not create a new graph thread.

4 | Clinic knowledge, model, and safety

The RAG retriever reads only clinic-approved documents from the configured filesystem directory. It returns evidence and source names; it does not query PostgreSQL.

The clinic-knowledge service should answer only when approved evidence supports the response, cite its source, and give an honest fallback when evidence is missing. Real model calls require a timeout; tests/CI use the deterministic fake model.

Emergency detection comes first. Emergency cases receive immediate emergency instructions and a staff flag; they bypass appointment booking and RAG. The assistant must not diagnose or recommend treatment.

5 | Initialization and runtime boundary

A one-off initializer applies the application schema and runs PostgresSaver.setup() before workflows depend on persistence. Fictional seed data is development-only.

PostgreSQL is the only database. RAG files and the external LLM provider are separate resources, not database connections.

IMPLEMENTATION STATUS | IMPORTANT

This PDF documents the target end-to-end architecture. In the current code, app/main.py registers the /health endpoint; the domain API/workflow integration is still in progress. Database schema/initialization and repository modules exist, but the diagram should not be read as proof that all components are wired together.

Keep the system backend-only: the clinic already has a chat client. Never place credentials or patient data in the diagram, repository, logs, or fixtures.